

ColdCast: Calibrated Cold-Start Prediction of AI System Performance

Yi Chen Kuo, Sudip Manchare, Jaideep Manupati, Akshada Shailesh Rane

Department of Computer Science, Virginia Tech

Alexandria, Virginia, USA

{alvin9840, akshada, jaideepm21, msudip}@vt.edu

Abstract—Evaluating an AI system across every item in every benchmark is expensive, while a single average accuracy obscures both system-specific capability and item-specific difficulty. We propose *ColdCast*, a deep-learning framework that predicts the probability that an AI system will answer a previously unseen evaluation item correctly without running that system on the target item. The project aligns with the Stanford AIMS Predictive AI Evaluation Competition, where inputs include AI-system metadata, item text and features, public response evidence, and a small budget of revealed labels for each test system. ColdCast encodes item semantics with a pretrained text encoder, represents AI systems through learned metadata embeddings, and fuses these signals to estimate calibrated probabilities of correctness. We ask whether semantic item representations, learned system representations, retrieval-based evidence, and five-label adaptation improve calibrated cold-start predictions beyond rate-based and standard neural baselines. We will use item-held-out validation to prevent leakage, evaluate Brier score as the primary metric, and report calibration error, reliability diagrams, cold-start performance, and ablation studies. The required project core is a text-plus-metadata fusion model with probability calibration; retrieval and adaptive label selection are extensions. The project will produce a reproducible pipeline, a competition-ready inference artifact, and an analysis of which information sources improve prediction reliability.

Index Terms—AI evaluation, calibration, cold-start prediction, deep learning, large language models, uncertainty estimation.

I. INTRODUCTION AND MOTIVATION

Evaluating modern AI systems is increasingly costly. Standard benchmarks comprise thousands of heterogeneous tasks, and evaluating every frontier model across every item requires prohibitive compute, runtime, and API expenses. Furthermore, aggregate benchmark accuracy is an incomplete summary: it conflates a capable model struggling on an adversarial prompt with a weak model failing on a trivial one. An effective alternative is predictive evaluation: estimating the probability that an AI system will answer an unseen item correctly without executing the target model.

The Stanford AIMS Predictive AI Evaluation Competition [1] formalizes this challenge. For each hidden subject (AI system) and item (task) pair, models must predict $\hat{p} = P(y = 1 \mid s, i)$, where $y = 1$ denotes outcome correctness. The setting enforces a benchmark-level cold start: test items originate from entirely held-out benchmarks absent from the public training pool. To make predictions, systems must leverage public response logs, system metadata, item semantics, and an adaptive budget of five revealed labels per test subject.

This project addresses open questions in multi-modal representation learning, out-of-distribution generalization, and uncertainty calibration. We propose ColdCast, an inductive deep learning framework that fuses semantic item encodings with structured system metadata embeddings to yield well-calibrated response probabilities on unseen benchmarks. Because submissions are evaluated using the Brier score, ColdCast is designed to optimize both ranking discrimination and probabilistic reliability, offering a scalable path toward efficient, surrogate-driven model evaluation.

A. Research Question and Objectives

Our research question is:

Can semantic item representations, learned AI-system representations, retrieval-based evidence, and five-label adaptation improve calibrated cold-start prediction of AI-system correctness beyond simple rate-based and standard neural baselines?

The project objectives are to: (1) build an item-held-out, leakage-safe validation protocol; (2) establish transparent statistical and neural baselines; (3) train a text-plus-metadata deep fusion model; (4) apply and evaluate probability calibration; (5) investigate retrieval and few-label adaptation when time permits; and (6) package a reproducible inference pipeline for a potential competition submission.

II. RELATED WORK AND RESEARCH CONTRIBUTION

A. Related Work

Recent research studies how to reduce the cost of AI evaluation and estimate capability from limited evidence. Truong et al. propose amortized model-based evaluation, using item content and adaptive testing to estimate performance more efficiently than exhaustive evaluation [2]. Zhuang et al. study computerized adaptive testing for measuring LLM cognitive ability with fewer questions [3]. These works motivate using item properties and a limited set of observed responses to predict performance on unobserved items.

Item Response Theory (IRT) is a classical approach for modeling latent ability and item difficulty. However, AI-evaluation settings may violate IRT assumptions because model populations can be small, heterogeneous, and rapidly changing. Jiang et al. identify reliability concerns for IRT estimates under such conditions [4]. Shang et al. combine contextual item embeddings with multidimensional IRT-style

modeling for evaluation on unseen questions and identify cross-scenario generalization as an open issue [5]. The AIMS competition provides a concrete benchmark requiring calibrated correctness probabilities for hidden system-item pairs under cold-start conditions [1].

B. Research Contribution

ColdCast will provide a controlled empirical analysis of complementary information sources for calibrated performance prediction:

- **Semantic item representations:** pretrained text embeddings plus structured item, task, and tool features.
- **System representations:** learned embeddings of model, provider, harness/version, reasoning effort, and other available settings.
- **Interaction modeling:** a fusion multilayer perceptron (MLP) and a neural factorization-style alternative.
- **Retrieval evidence:** features from semantically similar training items and comparable systems, constructed using training data only.
- **Calibration:** comparison of uncalibrated predictions and temperature-scaled probabilities.
- **Few-label adaptation:** an extension using five revealed labels and comparing random, uncertainty-based, and diversity-based selection strategies.

Our contribution is not limited to one leaderboard score. We will identify which information sources improve Brier score and calibration under item-held-out validation, measure the value of five revealed labels, and analyze systematic overconfidence and underconfidence.

III. DATA, BASELINES, AND EVALUATION PLAN

A. Data

We will use the public training release for the Stanford AIMS Predictive AI Evaluation Competition [1]. Each labeled observation corresponds to an AI-system and evaluation-item pair. Available inputs include system metadata (e.g., model, provider, harness/version, reasoning effort, dates, and other settings), item text and features, associated tools or interactors, and public historical correctness outcomes. The competition additionally provides a small number of revealed labels per test system.

Before training, we will create a data card documenting the available fields, dataset size, missing-value patterns, categorical cardinalities, text-length distributions, class balance, and applicable competition restrictions. We will follow official train/test boundaries and submission rules.

B. Leakage-Safe Validation

Cold-start generalization is the core challenge. Randomly splitting individual response rows would allow the same item to appear in both training and validation, creating an opportunity to memorize item difficulty. Our primary offline protocol will therefore hold out *entire items*: all response rows for a held-out item are excluded from training.

TABLE I
PLANNED BASELINES AND COMPARISON MODELS.

Method	Description
Constant 0.5	Predict $P(\text{correct}) = 0.5$ for all pairs.
Global rate	Predict the training-set global correctness rate.
Per-system rate	Smoothed historical correctness rate for each system.
Metadata-only	MLP using structured system metadata.
Text-only	MLP using item embeddings and structured item features.
Retrieval-only	Similarity-weighted statistics from training items/systems.
Fusion MLP	Learned fusion of item and system representations.
Neural factorization	Latent system-item interactions plus content features.

All preprocessing, categorical encodings, global or per-system rates, retrieval indexes, and feature transformations will be fit on the training partition only. Calibration parameters will be fit on a dedicated calibration split or through a cross-validation procedure that does not expose final-validation labels. We will report performance on relevant slices, including cold-start items, sparse-evidence systems, task categories, and available model/provider groups.

C. Baselines

Table I summarizes planned baselines. The progression enables us to quantify the value of prevalence, system history, semantic item information, metadata, retrieval, and learned interactions.

D. Evaluation Metrics and Ablations

Our primary metric is Brier score:

$$\text{Brier} = \frac{1}{N} \sum_{n=1}^N (\hat{p}_n - y_n)^2, \quad (1)$$

where $\hat{p}_n$ is the predicted probability and $y_n \in \{0, 1\}$ is the observed correctness label. Lower is better; a perfect predictor has Brier score zero, while a constant 0.5 prediction gives 0.25.

We will also report expected calibration error (ECE), reliability diagrams, binary cross-entropy, cold-start slice scores, and confidence intervals across repeated seeds or item-level bootstrap resampling when feasible. Our ablations will test: (1) item text beyond rate baselines, (2) metadata beyond item text, (3) fusion versus single-modality models, (4) retrieval features beyond fusion, (5) temperature scaling, and (6) five-label adaptation and label-selection policies if the extension stage is reached.

IV. PROPOSED DEEP-LEARNING APPROACH AND SYSTEM DESIGN

A. System Overview

For an AI system s and an evaluation item i , ColdCast estimates:

$$\hat{p}_{s,i} = \sigma(f_{\theta}(\mathbf{z}_i, \mathbf{u}_s, \mathbf{r}_{s,i}, \mathbf{a}_s)), \quad (2)$$

ColdCast workflow	
Item text	→ pretrained text encoder → item embedding
Item/task/tool features	→ feature encoder
System metadata	→ categorical embeddings → system embedding
Training-only semantic index	→ retrieval evidence (extension)
Five revealed labels	→ adaptation vector (extension)
All features	→ fusion MLP / factorization head → $P(\text{correct})$
Validation calibration	→ temperature-scaled probability

Fig. 1. Proposed ColdCast system design. The core system uses item and system representations with a fusion MLP. Retrieval and five-label adaptation are controlled extensions.

where $\mathbf{z}_i$ is an item representation, $\mathbf{u}_s$ is a system representation, $\mathbf{r}_{s,i}$ contains optional retrieval features, $\mathbf{a}_s$ is an optional adaptation vector from revealed labels, f_θ is a neural prediction head, and σ is the sigmoid function.

B. Item and System Encoders

Each item will be encoded with a pretrained sentence or text encoder. We will begin with frozen embeddings to reduce compute requirements and make the contribution of the prediction model easier to isolate. Structured item features such as task category, item type, tool/interactor availability, and length-related features when available, will be concatenated with the text representation and projected into $\mathbf{z}_i$.

Each AI system will be represented by learned embeddings for categorical metadata, including model identity, provider, harness/version, and reasoning effort. Numeric or temporal features will be normalized and projected through a small feed-forward network. These components form $\mathbf{u}_s$. Unknown-category handling, regularization, and smoothed-rate comparisons will reduce overfitting to rare system configurations.

C. Fusion, Retrieval, and Calibration

The required core prediction model concatenates $\mathbf{z}_i$ and $\mathbf{u}_s$, optionally adds element-wise interactions, and feeds the result to an MLP with dropout and a sigmoid output. We will train with binary cross-entropy and compare direct Brier-loss training if stable. A neural factorization-style head will be considered as an alternative for representing latent system-item interactions.

For the retrieval extension, we will build a semantic index using training items only. The module will derive similarity-weighted neighbor statistics, including correctness rates on comparable items and cross-system performance summaries. Retrieval-only predictions will be compared to fusion models augmented with retrieval features.

The core model’s probabilities will be calibrated using temperature scaling fit on a separate calibration split. For the few-label extension, the five revealed labels will produce a compact system-specific adaptation vector through summary features and similarity-weighted evidence. If feasible, we will compare random label selection with uncertainty-based and diversity-based policies under the competition rules.

TABLE II
PROJECT SCHEDULE AND MILESTONES.

Period	Milestone and outputs
Sep. 17–22	Finalize proposal, inspect data, define item-held-out split, create repository and experiment tracker.
Sep. 23–Oct. 6	Build data pipeline, data card, leakage checks, and constant/global/per-system baselines.
Oct. 7–20	Train text and metadata encoders plus fusion MLP; produce initial Brier-score table and reliability plot for Checkpoint I.
Oct. 21–Nov. 3	Compare fusion, factorization, and retrieval models; run main ablations.
Nov. 4–10	Complete calibration and error analysis; prepare Checkpoint II and 15-minute presentation.
Nov. 11–20	Implement feasible few-label extension; package and test competition submission artifacts.
Nov. 21–Dec. 8	Finalize figures, demo/video, reproducibility review, and final presentation.
Dec. 9–11	Complete IEEE-style final report and repository cleanup.

TABLE III
PROPOSED TEAM RESPONSIBILITIES.

Role	Primary responsibilities
Team leader / integration lead: Sudip	Coordinate schedule, repository, experiment tracking, report integration, and final packaging.
Data and validation lead: Yi Chen	Inspect data; implement preprocessing, item-held-out split, leakage tests, and rate baselines.
Modeling lead: Jaideep	Implement item/system encoders, fusion MLP, factorization model, and training configurations.
Retrieval and adaptation lead: Akshada	Build semantic retrieval features; investigate five-label adaptation and label selection.
Evaluation and presentation: Collective effort	Produce metrics, calibration plots, ablations, error analysis, project webpage, and demo materials.

D. Implementation and Reproducibility

We will implement ColdCast in Python using PyTorch and standard scientific-computing libraries. We will version configurations, fix random seeds, log all metrics, save checkpoints, and provide reproducible training, evaluation, and inference scripts. The initial frozen-embedding system should require modest GPU resources; the Virginia Tech computing environment will be used for experiments as needed.

V. PROJECT PLAN, TEAM ROLES, AND DELIVERABLES

A. Schedule

The project follows an incremental eight-stage progression from initial pipeline construction to final competition packaging and reporting. Table II summarizes the comprehensive project schedule, target timelines, and core deliverables.

B. Team Roles

To ensure accountability and modular development, project responsibilities are distributed across four specialized functional areas. Table III outlines the primary roles, technical ownership, and deliverable commitments assigned to each team member.

C. Deliverables and Scope Control

The final deliverables will include a reproducible code repository, preprocessing and leakage-safe split scripts, training/evaluation/inference code, trained artifacts, Brier and calibration tables, reliability diagrams, cold-start and failure-case analyses, a project webpage, a demonstration video, and a final report. If competition-ready, we will also package the required `model.py` and optional `labeling.py` artifacts.

The required scope is: item-held-out validation, rate baselines, text-only and metadata-only models, fusion MLP, Brier-score evaluation, temperature scaling, and calibration/error analysis. Neural factorization and training-only retrieval are strong extensions. Five-label adaptation, active selection, and a final competition submission are stretch goals.

VI. CONCLUSION

ColdCast addresses a timely problem in AI evaluation: estimating the probability that an AI system will correctly answer a previously unseen item without executing that system on the item. The project combines semantic item representation, system metadata modeling, calibrated probabilistic prediction, and leakage-safe cold-start evaluation. Its staged design ensures a feasible core contribution while preserving meaningful extensions in retrieval and few-label adaptation. By reporting both Brier score and calibration analyses, the project will evaluate not only whether predictions are useful but also whether their confidence is trustworthy.

REFERENCES

- [1] Stanford AI Measurement Science, “Predictive AI Evaluation Competition,” 2026. [Online]. Available: <https://aimslab.stanford.edu/competition/>
- [2] C. Truong et al., “Reliable and Efficient Amortized Model-based Evaluation,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [3] J. Zhuang et al., “Efficiently Measuring the Cognitive Ability of LLMs: An Adaptive Testing Perspective,” arXiv:2306.10512, 2023.
- [4] Y. Jiang et al., “Can We Trust Item Response Theory for AI Evaluation?,” arXiv:2607.15190, 2026.
- [5] Y. Shang, Z. Tang, and X. He, “LLM Evaluation on Unseen Questions,” arXiv:2608.22295, 2026.

APPENDIX

A. Mathematical Formulation of Optimization Objectives

During offline training, network parameters θ are optimized using binary cross-entropy loss with L_2 weight regularization:

$$\mathcal{L}_{\text{BCE}}(\theta) = -\frac{1}{N} \sum_{n=1}^N [y_n \log \hat{p}_n + (1 - y_n) \log(1 - \hat{p}_n)] + \lambda \|\theta\|_2^2. \quad (3)$$

For post-hoc calibration, temperature parameter $T > 0$ and intercept b are determined over a calibration set $\mathcal{D}_{\text{cal}}$ by minimizing negative log-likelihood:

$$\min_{T, b} - \sum_{(s, i, y) \in \mathcal{D}_{\text{cal}}} \left[y \log \sigma \left(\frac{z_{s, i}}{T} + b \right) + (1 - y) \log \left(1 - \sigma \left(\frac{z_{s, i}}{T} + b \right) \right) \right]. \quad (4)$$

B. Competition Runtime Specification

The competition runtime operates under the following constraints, which ColdCast’s build system enforces:

- **Submission Archive:** Flat ZIP structure containing `model.py` (defining `predict(input, labeled)`), optional `labeling.py` (defining `acquisition_function(input)`), and `models.txt`.
- **Input Interface:** `input` is a 2-element list `[subject, item]` of string dictionaries. `labeled` contains up to five entries of `[[subject, item], label]`.
- **Output Constraints:** `predict()` must return a native scalar Python float in `[0.0, 1.0]`. Values outside this range or NaN triggers failure code 42.
- **Sandbox Constraints:** No runtime Internet access. Pre-trained Hugging Face transformer weights are prefetched via `models.txt` into local cache storage.